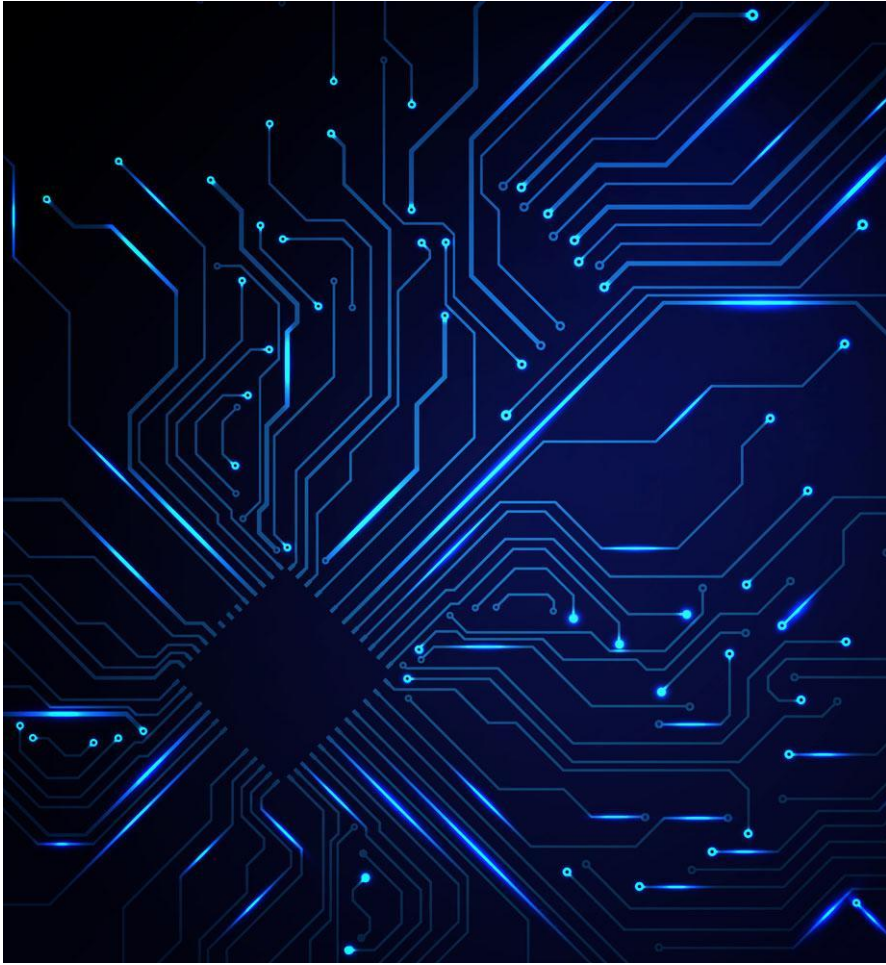




2026/05/18

實驗十一

Maze Solver Hardware Design

姓名：陳篁瑋 學號：01257024

班級：資工 3A

E-mail：henry246813579@gmail.com

注意

1. 繳交時一律轉 PDF 檔
2. 繳交期限為
隔週三上午九點
3. 一人繳交一份
4. 檔名：學號_HW?.pdf
檔名請按照作業檔名格式進行填寫
未依照格式不予批改

Maze Solver Hardware Design

■ 實驗說明：

1. 請設計一個硬體架構，在 17×17 的網格迷宮中尋找路徑。迷宮包含路徑、牆壁、陷阱與人質。目標是從 (0, 0) 出發，救出所有人質並避開陷阱，最後抵達目的地 (16, 16)。

2. Maze 輸入：

訊號	說明
rst_n	系統重置訊號，為 0 時重置系統。
clk	系統時脈訊號，頻率為 50MHz。
landform [1:0]	當前網格的地形類型編碼。
valid	當 landform 資料有效時發出的訊號。

3. Maze 輸出：

訊號	說明
ready	模組準備好接收下一個地圖資料的訊號。
action [2:0]	下一步移動方向： 000=右, 001=下, 010=左, 011=上, 100=停頓。
finish	抵達終點且救出所有人質時發出的結束訊號。

4. 根據以下做評分：

BATCH_MODE = 0 :

1. 波行邏輯正確
2. transcript 無 FAILURE
3. 請錄製軟體(animation_result.py)執行過程，確保人質都有救出、踩到陷阱必須停一個 clk(陷阱不會消失)、順利抵達終點且無任何不合法行為(走到迷宮外或走到牆上)

BATCH_MODE = 1 :

1. transcript 無 FAILURE

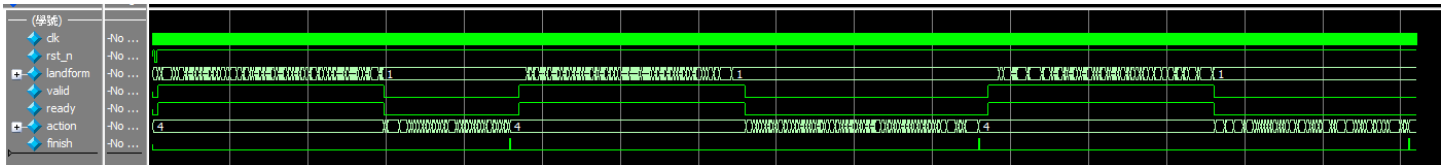
補充說明:

BATCH_MODE 請自行在 tb 更改

```
1 // 修改vsim => vsim -c -sv_seed random work.testbench -do "run -all;"
2 `timescale 1ns/10ps
3
4 module testbench;
5
6 // Batch toggle: 1 for input_total.txt, 0 for individual files
7 parameter BATCH_MODE = 0;
8 // Max mazes to process
```

■ 波型圖參考

完整波型:



細節:

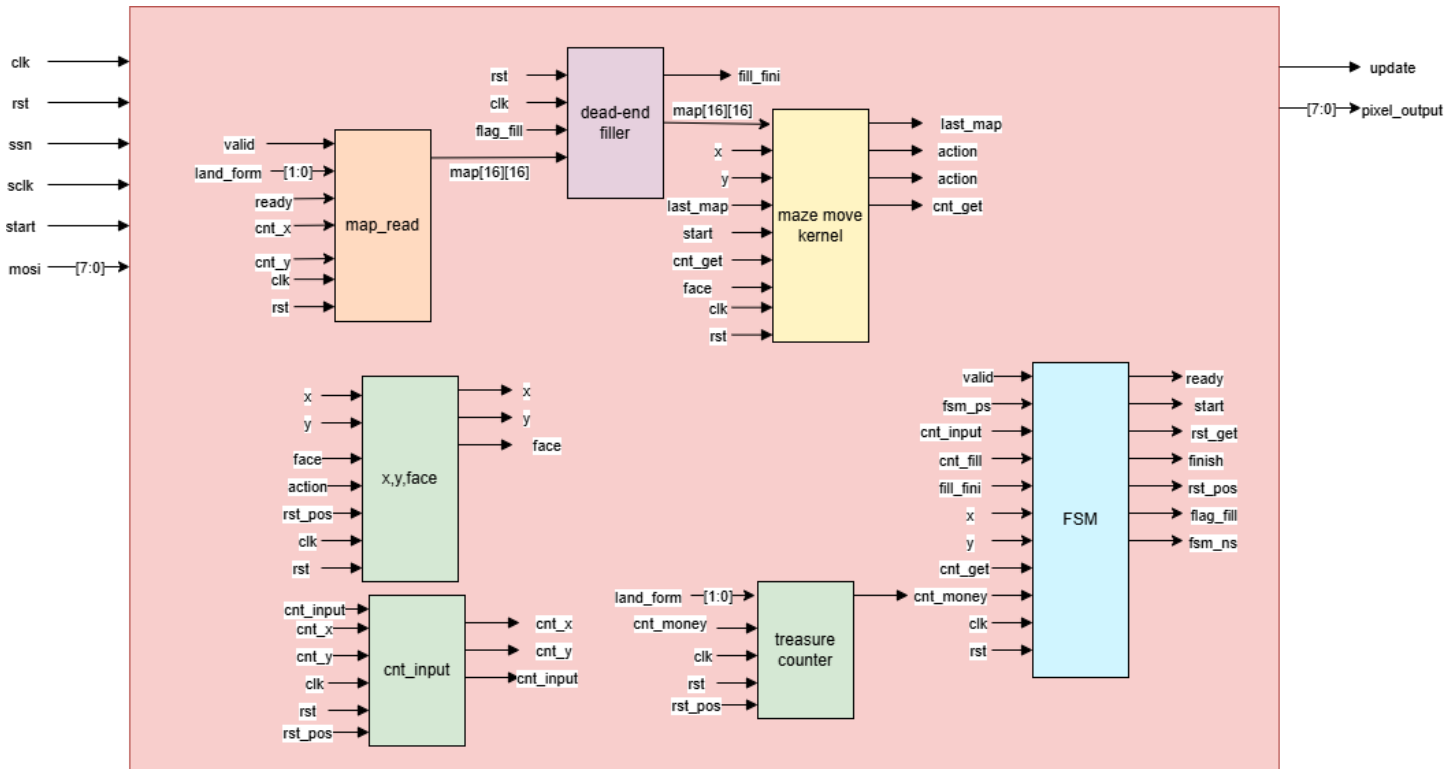


finish

■ Transcript 視窗參考

```
# Transmitting Maze 14891 (Test 4891/5000)...
# Transmitting Maze 14901 (Test 4901/5000)...
# Transmitting Maze 14911 (Test 4911/5000)...
# Transmitting Maze 14921 (Test 4921/5000)...
# Transmitting Maze 14931 (Test 4931/5000)...
# Transmitting Maze 14941 (Test 4941/5000)...
# Transmitting Maze 14951 (Test 4951/5000)...
# Transmitting Maze 14961 (Test 4961/5000)...
# Transmitting Maze 14971 (Test 4971/5000)...
# Transmitting Maze 14981 (Test 4981/5000)...
# Transmitting Maze 14991 (Test 4991/5000)...
# =====
# All 5000 maps tested.
# Total SUCCESS : 5000 / 5000
# Total FAILURE : 0
# Total cycles : 2496412
# Average cycles: 499.28
# =====
# ** Note: $stop : C:/Users/Alex/Downloads/00_TESTBED/simulation/tb/testbench.sv(170)
# Time: 25514175 ns Iteration: 1 Instance: /testbench
# Break in Module testbench at C:/Users/Alex/Downloads/00_TESTBED/simulation/tb/testbench.sv line 170
```

■ 系統架構圖或方塊圖說明(請繪製自己設計的硬體架構圖並說明)
繪圖請善用 Draw.io



- FSM

它負責接收來自各個模組的狀態回報（如 `cnt_input` 讀完沒、`fill_fini` 填補完沒、`x,y` 走到終點沒），並根據這些條件切換系統的四大階段：INIT → INPUT → FILL → MOVE → OVER。還要發出控制訊號（如 `ready` 允許讀取、`flag_fill` 啟動填補、`start` 啟動移動、`rst_pos` 回合歸零）給其他模組。

- map_read

負責將外部傳入的地形資料（`land_form`），依照座標（`cnt_x`, `cnt_y`）存入 `map[16][16]` 中。

- dead-end filler

收到 `flag_fill` 後不斷掃描 `map`，把死路填起來。當偵測到地圖不再變動或達到次數上限時，輸出 `fill_fini` 通知 FSM 預處理完畢。

- maze move kernel

根據目前的座標（`x`, `y`）、`face` 以及 `map`，計算出這回合該往哪裡走，並輸出 `action`。同時負責判斷是否吃到寶藏並更新 `cnt_get`。

- `x`, `y`, `face`

負責接收 `kernel` 決定的 `action`，並在下一個 Clock 更新實際的 `x` 座標、`y` 座標以及面朝方向 `face`。

- `cnt_input`

產生 cnt_x 和 cnt_y 交給 map_read 當作寫入的地址，並回報 cnt_input 給 FSM 判斷 289 格是否讀取完畢。

- treasure counter

一開始輸入地圖時，只要看到 land_form == 3 就加一，最後算出這張地圖的總寶藏數 (cnt_money)，當作通過條件。

■ 系統架構程式碼與程式碼說明

截圖請善用 win+shift+S

```

1 module escape(
2     input logic clk,
3     input logic rst_n, // active-low reset
4     input logic [1:0] landform, // 00 wall, 01 path, 10 trap, 11 hostage
5     input logic valid, // 成功傳送就 valid
6     output logic ready, // 讀取資料
7     output logic [2:0] action, // 0:R,1:D,2:L,3:U,4:STALL
8     output logic finish // high when (16,16) reached AND all hostages rescued
9 );
10
11 //MAZE_move_kernel
12 logic [3:0] map [16:0][16:0];
13 logic [4:0] cnt_x;
14 logic [4:0] cnt_y;
15 logic [10:0] cnt_input; //算輸入了幾個位置了
16 logic [9:0] cnt_money;
17 logic [9:0] cnt_get;
18 logic rst_get;
19 logic [4:0] x;
20 logic [4:0] y;
21 logic start;
22 logic [1:0] face; //right = 0 down = 1 left = 2 up = 3
23 logic [1:0] last_map; //00 wall, 01 path, 10 trap, 11 hostage
24 logic rst_pos;
25 logic [8:0] cnt_fill;
26 logic flag_fill;
27 logic fill_fini;
28 integer i;
29 integer j;
30 always_ff@(posedge clk)
31 begin
32     if(~rst_n) begin
33         cnt_x <= 0;
34         cnt_y <= 0;
35         cnt_input <= 0;
36         cnt_money <= 0;
37         x <= 0;
38         y <= 0;
39         face <= 0;
40         last_map <= 0;
41         action <= 4;
42         cnt_fill <= 0;
43         fill_fini <= 1;
44     end
45     else begin
46         if(valid && cnt_input < 289 && ready) begin
47             map[cnt_x][cnt_y] <= landform;
48             if(landform == 3)
49                 cnt_money <= cnt_money + 1;
50             cnt_input <= cnt_input + 1;
51             if(cnt_x == 16) begin
52                 cnt_x <= 0;
53                 if(cnt_y == 16)
54                     cnt_y <= 0;
55                 else
56                     cnt_y <= cnt_y + 1;
57             end
58             else
59                 cnt_x <= cnt_x + 1;
60         end
61         if(rst_pos) begin
62             cnt_money <= 0;
63             action <= 4;
64             cnt_input <= 0;
65             x <= 0;
66             y <= 0;
67             face <= 0;
68             last_map <= 0;
69             cnt_x <= 0;
70             cnt_y <= 0;
71             cnt_fill <= 0;
72             fill_fini <= 1;
73         end
74         if(flag_fill) begin
75             cnt_fill <= cnt_fill + 1;
76             fill_fini <= 0;
77             for (i=0; i<17; i=i+1) begin
78                 for (j=0; j<17; j=j+1) begin
79                     if ((map[i][j] == 1 || map[i][j] == 2) && !(i==0 && j==0) && !(i==16 && j==16)) begin
80                         if ((j == 0) ? 1'b1 : (map[i][j]>0 ? j-1 : 0) == 0) +
81                             ((j == 16) ? 1'b1 : (map[i][j]<16 ? j+1 : 16) == 0) +
82                             ((i == 0) ? 1'b1 : (map[i]>0 ? i-1 : 0)[j] == 0) +
83                             ((i == 16) ? 1'b1 : (map[i]<16 ? i+1 : 16)[j] == 0) >= 3 ) begin
84                             fill_fini <= 1;
85                             map[i][j] <= 0;
86                         end
87                     end
88                 end
89             end
90         end
91         if(start) begin
92             case(face)
93                 0:begin
94                     if(map[x][y-1]!=0&&y!=0) begin//up
95                         if(last_map == 2) begin
96                             action <= 4;
97                             last_map <= 0;
98                         end

```

```

99         else begin
100             face    <= 3;
101             action  <= 3;
102             last_map <= map[x][y-1];
103             y        <= y-1;
104             if(map[x][y-1]==3) begin
105                 cnt_get    <= cnt_get +1;
106                 map[x][y-1] <= 1;
107             end
108         end
109     end
110     else if(map[x+1][y]!=0&&x!=16) begin//right
111         if(last_map == 2) begin
112             action <= 4;
113             last_map <= 0;
114         end
115         else begin
116             face    <= 0;
117             action  <= 0;
118             last_map <= map[x+1][y];
119             x        <= x+1;
120             if(map[x+1][y]==3)begin
121                 cnt_get    <= cnt_get +1;
122                 map[x+1][y] <= 1;
123             end
124         end
125     end
126     else if(map[x][y+1]!=0&&y!=16) begin//down
127         if(last_map == 2) begin
128             action <= 4;
129             last_map <= 0;
130         end
131         else begin
132             face    <= 1;
133             action  <= 1;
134             last_map <= map[x][y+1];
135             y        <= y+1;
136             if(map[x][y+1]==3)begin
137                 cnt_get    <= cnt_get +1;
138                 map[x][y+1] <= 1;
139             end
140         end
141     end
142     else if(map[x-1][y]!=0&&x!=0) begin//left
143         if(last_map == 2) begin
144             action <= 4;
145             last_map <= 0;
146         end
147         else begin
148             face    <= 2;
149             action  <= 2;
150             last_map <= map[x-1][y];
151             x        <= x-1;
152             if(map[x-1][y]==3)begin
153                 cnt_get    <= cnt_get +1;
154                 map[x-1][y] <= 1;
155             end
156         end
157     end
158 end
159 1:begin
160     if(map[x+1][y]!=0&&x!=16) begin//right
161         if(last_map == 2) begin
162             action <= 4;
163             last_map <= 0;
164         end
165         else begin
166             face    <= 0;
167             action  <= 0;
168             last_map <= map[x+1][y];
169             x        <= x+1;
170             if(map[x+1][y]==3)begin
171                 cnt_get    <= cnt_get +1;
172                 map[x+1][y] <= 1;
173             end
174         end
175     end
176     else if(map[x][y+1]!=0&&y!=16) begin//down
177         if(last_map == 2) begin
178             action <= 4;
179             last_map <= 0;
180         end
181         else begin
182             face    <= 1;
183             action  <= 1;
184             last_map <= map[x][y+1];
185             y        <= y+1;
186             if(map[x][y+1]==3)begin
187                 cnt_get    <= cnt_get +1;
188                 map[x][y+1] <= 1;
189             end
190         end
191     end
192     else if(map[x-1][y]!=0&&x!=0) begin//left
193         if(last_map == 2) begin
194             action <= 4;
195             last_map <= 0;
196         end
197     end

```

```

197     else begin
198         face    <= 2;
199         action  <= 2;
200         last_map <= map[x-1][y];
201         x       <= x-1;
202         if(map[x-1][y]==3)begin
203             cnt_get    <= cnt_get +1;
204             map[x-1][y] <= 1;
205         end
206     end
207 end
208 else if(map[x][y-1]!=0&&y!=0) begin//up
209     if(last_map == 2) begin
210         action <= 4;
211         last_map <= 0;
212     end
213     else begin
214         face    <= 3;
215         action  <= 3;
216         last_map <= map[x][y-1];
217         y       <= y-1;
218         if(map[x][y-1]==3)begin
219             cnt_get    <= cnt_get +1;
220             map[x][y-1] <= 1;
221         end
222     end
223 end
224 end
225 2:begin
226     if(map[x][y+1]!=0&&y!=16) begin//down
227         if(last_map == 2) begin
228             action <= 4;
229             last_map <= 0;
230         end
231         else begin
232             face    <= 1;
233             action  <= 1;
234             last_map <= map[x][y+1];
235             y       <= y+1;
236             if(map[x][y+1]==3)begin
237                 cnt_get    <= cnt_get +1;
238                 map[x][y+1] <= 1;
239             end
240         end
241     end
242     else if(map[x-1][y]!=0&&x!=0) begin//left
243         if(last_map == 2) begin
244             action <= 4;
245             last_map <= 0;
246         end
247         else begin
248             face    <= 2;
249             action  <= 2;
250             last_map <= map[x-1][y];
251             x       <= x-1;
252             if(map[x-1][y]==3)begin
253                 cnt_get    <= cnt_get +1;
254                 map[x-1][y] <= 1;
255             end
256         end
257     end
258     else if(map[x][y-1]!=0&&y!=0) begin//up
259         if(last_map == 2) begin
260             action <= 4;
261             last_map <= 0;
262         end
263         else begin
264             face    <= 3;
265             action  <= 3;
266             last_map <= map[x][y-1];
267             y       <= y-1;
268             if(map[x][y-1]==3)begin
269                 cnt_get    <= cnt_get +1;
270                 map[x][y-1] <= 1;
271             end
272         end
273     end
274     else if(map[x+1][y]!=0&&x!=16) begin//right
275         if(last_map == 2) begin
276             action <= 4;
277             last_map <= 0;
278         end
279         else begin
280             face    <= 0;
281             action  <= 0;
282             last_map <= map[x+1][y];
283             x       <= x+1;
284             if(map[x+1][y]==3)begin
285                 cnt_get    <= cnt_get +1;
286                 map[x+1][y] <= 1;
287             end
288         end
289     end
290 end
291 3:begin
292     if(map[x-1][y]!=0&&x!=0) begin//left
293         if(last_map == 2) begin
294             action <= 4;

```

```

295         last_map <= 0;
296     end
297 else begin
298     face <= 2;
299     action <= 2;
300     last_map <= map[x-1][y];
301     x <= x-1;
302     if(map[x-1][y]==3)begin
303         cnt_get <= cnt_get +1;
304         map[x-1][y] <= 1;
305     end
306 end
307 end
308 else if(map[x][y-1]!=0&&y!=0) begin//up
309     if(last_map == 2) begin
310         action <= 4;
311         last_map <= 0;
312     end
313     else begin
314         face <= 3;
315         action <= 3;
316         last_map <= map[x][y-1];
317         y <= y-1;
318         if(map[x][y-1]==3)begin
319             cnt_get <= cnt_get +1;
320             map[x][y-1] <= 1;
321         end
322     end
323 end
324 else if(map[x+1][y]!=0&&x!=16) begin//right
325     if(last_map == 2) begin
326         action <= 4;
327         last_map <= 0;
328     end
329     else begin
330         face <= 0;
331         action <= 0;
332         last_map <= map[x+1][y];
333         x <= x+1;
334         if(map[x+1][y]==3)begin
335             cnt_get <= cnt_get +1;
336             map[x+1][y] <= 1;
337         end
338     end
339 end
340 else if(map[x][y+1]!=0&&y!=16) begin//down
341     if(last_map == 2) begin
342         action <= 4;
343         last_map <= 0;
344     end
345     else begin
346         face <= 1;
347         action <= 1;
348         last_map <= map[x][y+1];
349         y <= y+1;
350         if(map[x][y+1]==3)begin
351             cnt_get <= cnt_get +1;
352             map[x][y+1] <= 1;
353         end
354     end
355 end
356 end
357 endcase
358 end
359 if(~rst_n || rst_get)
360     cnt_get <= 0;
361 end
362 end
363
364 //FSM
365 typedef enum{ INIT, INPUT_DATA, MOVE, OVER, FILL} FSM_STATE;
366 FSM_STATE fsm_ns,fsm_ps;
367
368 always_ff@(posedge clk) begin
369     if(~rst_n)
370         fsm_ps <= INIT;
371     else
372         fsm_ps <= fsm_ns;
373 end
374
375 always_comb
376 begin
377     fsm_ns = fsm_ps;
378     ready = 0;
379     start = 0;
380     rst_get = 0;
381     finish = 0;
382     rst_pos = 0;
383     flag_fill = 0;
384     case(fsm_ps)
385     INIT:
386     begin
387         if(valid)
388             fsm_ns = INPUT_DATA;
389             rst_pos = 1;
390     end
391     INPUT_DATA:
392     begin

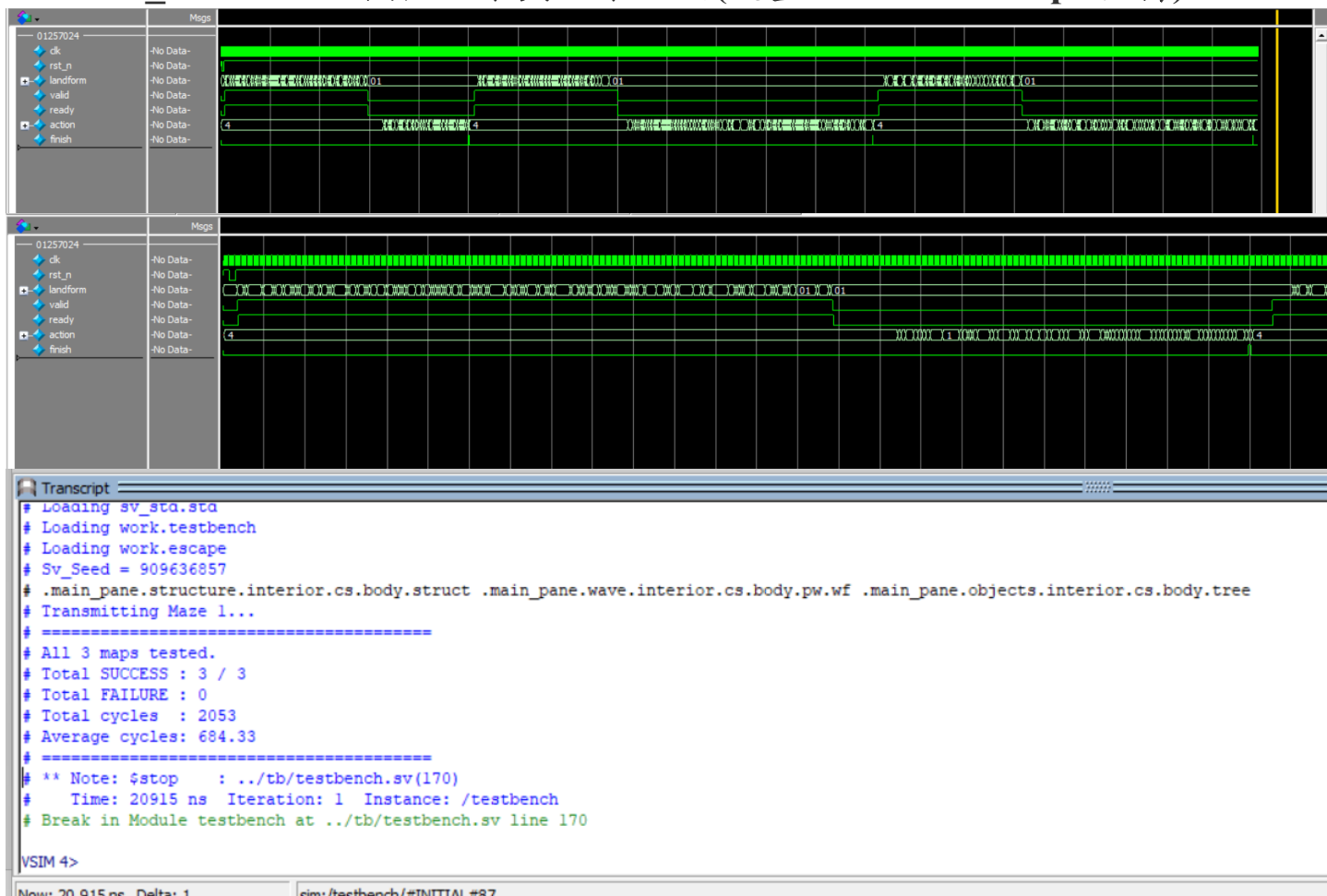
```

```

393         ready = 1;
394         if(cnt_input == 289) begin
395             rst_get = 1;
396             fsm_ns = FILL;
397         end
398     end
399     FILL:
400     begin
401         flag_fill = 1;
402         if(cnt_fill > 0 && fill_fini == 0)
403             fsm_ns = MOVE;
404     end
405     MOVE:
406     begin
407         start = 1;
408         if(x==16 && y==16 && cnt_get >= cnt_money)
409             fsm_ns = OVER;
410     end
411     OVER:
412     begin
413         finish = 1;
414         fsm_ns = INIT;
415     end
416 endcase
417 end
418 endmodule
419
420

```

■ BATCH_MODE = 0 模擬結果與結果說明(波型 以及 Transcript 視窗)：



■ BATCH_MODE = 1 結果說明(Transcript 視窗)：

```
# Transmitting Maze 14911 (Test 4911/5000)...
# Transmitting Maze 14921 (Test 4921/5000)...
# Transmitting Maze 14931 (Test 4931/5000)...
# Transmitting Maze 14941 (Test 4941/5000)...
# Transmitting Maze 14951 (Test 4951/5000)...
# Transmitting Maze 14961 (Test 4961/5000)...
# Transmitting Maze 14971 (Test 4971/5000)...
# Transmitting Maze 14981 (Test 4981/5000)...
# Transmitting Maze 14991 (Test 4991/5000)...
# =====
# All 5000 maps tested.
# Total SUCCESS : 5000 / 5000
# Total FAILURE : 0
# Total cycles : 2385825
# Average cycles: 477.17
# =====
# ** Note: $stop      : ../tb/testbench.sv(170)
#    Time: 24408305 ns Iteration: 1 Instance: /testbench
# Break in Module testbench at ../tb/testbench.sv line 170
V$SIM 6>
```

■ 結論與心得：

這次的迷宮在寫移動的邏輯時，會很複雜，遇到很多判斷的東西，要一直跑 python 的移動程式，然後去 debug。